Department of Computing
Xi'an Jiaotong-Liverpool University

Tutorial/Lab 1 – OOP Review 1

Aim

This tutorial/lab aims to review basic object-oriented programming concepts, such as arrays, classes, objects, and arrays of objects.

Tips

Create a good test set first, which can cover all kinds of cases of input values. This will be helpful to test your code later after you write your solution, but also in understanding the problem statement before you start solving the problem.

Exercises

Exercise 1.1: In-place Reverse

- The reverse method in the lecture reverses an array of integers by copying it to a new array and returning it.
- Write a method `reverseInPlace` that reverses the array of doubles passed in the argument and returns nothing.
- After that, write a test program that prompts the user to enter arbitrary numbers, invokes the method to reverse the numbers and displays the array.
- Your test program should first prompt the user to enter the array size.
- Test the method on all kinds of input array.

Exercise 1.2: Deciding Four Consecutive Numbers

- Write the following method that tests whether the array has four consecutive numbers with the same value:
  `public static boolean isConsecutiveFour(int[] values)`
- After that, write a test program that prompts the user to enter a series of integers and displays it if the series contains four consecutive numbers with the same value.
- Your program should first prompt the user to enter the input size—i.e., the number of values in the series.
- Test the method on all kinds of input values.

Exercise 1.3: Longest Run of Equal Values

- Write the following method that finds the length of the longest consecutive run of identical values:
  `public static int longestRun(int[] values)`
- Example: for `[3, 3, 3, 2, 2, 9]`, the method returns `3`.
- After that, write a test program that:
  - Prompts the user to enter the input size.
  - Reads the integers.
  - Displays the length of the longest run.
- Test with:
  - All values different
  - All values the same
  - Runs at the beginning/end
  - Empty input (size 0)

Exercise 1.4: Stopwatch to Measure Running Time

- Design a class named `StopWatch`. The class should contain:
  - Private data fields `startTime` and `endTime` with getter methods.
  - An empty constructor that initializes `startTime` and `endTime` with the current time.
  - A method named `start()` that resets the `startTime` to the current time.
  - A method named `stop()` that sets the `endTime` to the current time.
  - A method named `getElapsedTime()` that returns the elapsed time (the difference between stop and start time) for the stopwatch in milliseconds.
- Note that you can use `System.currentTimeMillis()` to obtain the current time.
- Write a test program that measures the execution time of sorting 100,000 numbers using ***Selection Sort*** discussed during the lecture.

Exercise 1.5: Student Records

- Create a class `Student` to represent students in an Advanced OOP course.
- Each student object should represent a first name, last name, email address, and group number.
- Include a `toString()` method that returns a string representation of a student and a `less()` method that compares two students by their group numbers.
- Write a test program that prints and compares `Student` objects.

Exercise 1.6: Book Records

- Create a class `Book` representing a library book:
  - Fields: `title`, `author`, `year`, `isbn`
- Include the following:
  - A `toString()` method that returns a readable one-line description.
  - A method that compares two books by year (earlier year is "smaller"):
    `public boolean less(Book other)`
- Write a program that:
  - Creates at least 4 `Book` objects.
  - Prints them to the console.
  - Compares a few pairs using `less()` and prints the results.

Exercise 1.7: Assign Grades

Write a program that reads student scores (assume the scores can be between 0 and 100), gets the best score, and then assigns grades based on the following scheme:

- Grade is A if `score` is >= the best score -10;
- Grade is B if `score` is >= the best score -20;
- Grade is C if `score` is >= the best score -30;
- Grade is D if `score` is >= the best score -40;
- Grade is F otherwise.

The program prompts the user to enter the total number of students, then prompts the user to enter all of the scores, and concludes by displaying the grades. Here is a sample run:

```
Enter the number of students: 4 ↵ Enter
Enter 4 scores: 40 55 70 58 ↵ Enter
Student 0 score is 40 and grade is C
Student 1 score is 55 and grade is B
Student 2 score is 70 and grade is A
Student 3 score is 58 and grade is B
```

Exercise 1.8: Stock Class

Design a class named `Stock` that contains:

- A `string` data field named `symbol` for the stock's symbol.
- A `string` data field named `name` for the stock's name.
- A `double` data field named `previousClosingPrice` that stores the stock price for the previous day.
- A `double` data field named `currentPrice` that stores the stock price for the current time.
- A constructor that creates a stock with the specified symbol and name.
- A method named `getChangePercent()` that returns the percentage changed from `previousClosingPrice` to `currentPrice`.

Draw the UML diagram for the class, then implement the class. Write a test program that creates a `Stock` object with the stock symbol `ORCL`, the name `Oracle Corporation`, and the previous closing price of `34.5`. Set a new current price to `34.35` and display the price-change percentage.